How I Built My Portfolio Website with Hugo from Scratch

March 17, 2026

I want to be upfront about something before we get started. When I first decided to build my portfolio website, I had no idea it would turn into the project it eventually became. What started as a single HTML file sitting in a folder on my laptop turned into a weeks-long process of building, breaking, fixing, and rebuilding. This post is the story of that entire journey, written honestly so that if you are going through something similar, you actually understand what happened and why.

Where It All Started

Every developer reaches a point where they need a portfolio. For me that moment came when I realized I had been working on AI projects for a while and had absolutely nothing online to show for it. No personal site, no blog, nothing. So I did what most people do in that situation. I opened a text editor and started writing a single HTML file.

That file had everything crammed into it. The styles were written directly in a <style> tag at the top of the page, the JavaScript was at the bottom in a <script> tag, and the content was all hardcoded HTML in the middle. It worked. It looked decent. But it had one big problem that I did not fully appreciate at the time.

I wanted to write blog posts.

The moment you want to add a blog to a static HTML website, you realize that hardcoded HTML is not going to cut it. Adding a new blog post would mean copying an entire HTML file, editing the content inside it, and manually updating an index page to link to it. That is not how anyone should spend their time.

Why I Chose Hugo

I looked at several options. There are plenty of static site generators out there and a few JavaScript-heavy frameworks that could technically do what I needed. I landed on Hugo for a few specific reasons.

First, Hugo is built in Go and it is genuinely fast. When you run hugo server, it rebuilds your entire site in milliseconds. When you are actively developing and refreshing the browser constantly, this matters more than you would think.

Second, Hugo uses plain Markdown files for content. That means every blog post I write is just a .md file sitting in a folder. No database, no CMS, no account to log into. Just files. I can open any post in a text editor, change a line, and push it to GitHub.

Third, Hugo has a clean concept called content sections. You define a content/blog/ folder and a content/projects/ folder and Hugo automatically knows how to list and render everything inside them. Adding a new blog post is as simple as dropping a new Markdown file into the right folder.

The trade-off with Hugo is that the learning curve around templates can be confusing at first. Hugo uses Go templates which have their own syntax, and when something goes wrong, the error messages are not always the most helpful. I learned this pretty quickly.

Setting Up the Project

After installing Hugo, I created a fresh project and started building out the folder structure. Hugo generates a skeleton for you when you run hugo new site portfolio but most of that skeleton is empty directories waiting to be filled.

The structure I ended up with was straightforward. The content folder holds all the Markdown files for blog posts and projects. The layouts folder holds the HTML templates that Hugo uses to render those Markdown files into actual pages. The static folder holds CSS, JavaScript, and any other files that should be served as-is without Hugo touching them.

The key insight with Hugo is the separation between content and presentation. Your blog post is just words in a Markdown file. Hugo's job is to take those words and wrap them in the right HTML template to produce a finished page. Once you internalize that separation, everything else starts to make sense.

The First Problem I Hit

About twenty minutes after I got the project set up, I ran hugo server and opened the browser. The HTML structure was there but the page looked completely broken. No styles. No fonts. Just unstyled text on a white background.

My first instinct was that something was wrong with the CSS file itself. I checked the file. The CSS was fine. I checked the link tag in the HTML template. It looked correct. I spent a while going back and forth before I finally realized what was happening.

In hugo.toml, the main configuration file, there is a setting called baseURL. I had set it to my intended production URL, something like https://ik-awais.github.io/. Hugo uses that value to construct all the paths to your static files. When you are running the site locally on localhost:1313, those paths point to the wrong place entirely, so the browser cannot find the CSS file.

The fix was simple. Change baseURL to just / during development. That one character was the cause of probably an hour of confusion.

Building the Design

Once the basic plumbing was working, I focused on the design. I wanted something that felt modern and distinctive rather than the typical developer portfolio look. Most portfolios I had seen used either a very minimal white design or a generic dark theme with blue accents. I wanted something with more personality.

I decided on a very dark background, almost black with a slight purple tint, and used a combination of cyan and purple as the accent colors. The typography choice was important to me as well. I picked Syne for

headings because it has a bold, geometric quality that reads as technical without being cold. For body text and code, I used JetBrains Mono, which is a font designed specifically for developers. Using a monospace font as the body font is unconventional but I think it gives the site a cohesive identity.

The hero section went through several iterations. I eventually landed on animated orbital rings — CSS circles that rotate around a center point using keyframe animations. It is a simple effect visually but it gives the hero section some life without being distracting.

For the scrolling tech stack section, I used two rows of technology icons that scroll infinitely in opposite directions. This is done entirely in CSS using @keyframes with translateX. The icons come from a CDN called Devicon which hosts color SVGs of virtually every programming language and framework.

Dark and Light Mode

Adding a light mode was one of the better decisions I made during this project. The approach I took was CSS custom properties, also known as CSS variables. You define all your colors as variables at the root level of your stylesheet and then when the user toggles the theme, you swap those variables. No duplicate stylesheets, no JavaScript injecting inline styles. Just one clean swap.

The challenge was making the light mode actually look good and not just be a washed-out version of the dark mode. My first attempt at a light theme looked terrible. I was using a warm off-white background and the text colors were based on opacity modifiers stacked on top of each other. Everything looked muddy.

The version that actually worked took inspiration from GitHub and VS Code. A cool grey background at #f0f2f5, near-black text at #111827, and explicit color values for every element rather than opacity tricks. Code blocks stayed dark even in light mode, which was an intentional design decision. Reading code on a dark background is genuinely easier and it creates a nice contrast on the page.

The theme preference is saved to localStorage so returning visitors see the same theme they left with.

The Blog and Table of Contents

Building the blog section was where I ran into the most interesting technical problems.

The first issue was getting blog content to actually render visibly on the page. I had a CSS class called .reveal that I was using throughout the site for scroll-triggered animations. The way it works is that elements start with opacity: 0 and when they scroll into the viewport, a JavaScript observer adds a class that transitions them to opacity: 1. I had applied this class to the blog content wrapper as well.

On most sections of the page this works fine because the elements are below the fold when the page loads, so the observer triggers correctly as you scroll down. On a blog post page though, the main content is at the top of the page and is already in the viewport the moment the page loads. The observer sometimes missed this and the content stayed invisible. The fix was to simply not apply the reveal animation to the main blog content. It should always be visible immediately.

The second problem was the table of contents. I wanted a sticky sidebar on the right side of every blog post

that shows all the headings and lets readers jump to any section. My first approach was to write JavaScript that reads the headings in the page and builds a navigation list dynamically. This seemed reasonable.

It did not work.

The problem was timing. The JavaScript in main.js loads and runs before the inline script that was building the table of contents. By the time my table of contents script tried to register a DOMContentLoaded listener, that event had already fired. The script ran, found no headings, and produced an empty navigation.

I tried several workarounds before I found the right solution. Hugo has a built-in feature called .TableOfContents. When you use it in a template, Hugo reads your Markdown headings at build time and generates the entire table of contents as HTML before the page is ever served to a browser. No JavaScript, no timing issues, no DOM querying. The table of contents is just there in the HTML, ready to go.

This was a good reminder that the best solution is often the one that does not rely on client-side JavaScript to do something that can be done at build time.

The Contact Form

A portfolio without a contact form is just a display case. I wanted people to actually be able to reach me.

I used Formspree for the backend. Formspree gives you an endpoint URL that you POST form data to and they handle delivering it to your email. For a static site with no backend server, this is the cleanest option available.

The first version of the form used fetch() with JSON as the content type. This worked until I tried to enable reCAPTCHA on my Formspree account to filter out spam. Formspree's reCAPTCHA integration only works with HTML form submissions using multipart/form-data. JSON submissions get rejected. I eventually disabled the reCAPTCHA entirely and relied on client-side protection instead.

The client-side protection I built has three layers. First, deep email validation that goes beyond checking for an at-sign. It checks that the domain looks real, that the TLD is between two and twelve characters, that there are no consecutive dots, and it maintains a blocklist of over fifty known disposable email services like Mailinator and Guerrilla Mail. Second, a honeypot field. This is a form input that is completely hidden from real users using CSS but is present in the HTML. Bots that automatically fill in every field on a page will fill in the honeypot, and when the JavaScript sees that field has a value, it discards the submission silently. Third, rate limiting. A user can submit a maximum of three times per session with at least sixty seconds between each submission.

This combination is more effective than reCAPTCHA for my purposes because it creates no friction for legitimate users while still blocking the vast majority of automated spam.

Deploying with GitHub Actions

Once the site was working locally, I needed to get it onto the internet. I chose GitHub Pages because my repository is already hosted on GitHub and the integration is straightforward.

The way GitHub Pages works with Hugo is that you push your source code and a GitHub Actions workflow automatically builds the site and deploys the output. You never manually touch the public directory or commit build artifacts. The source code and the deployed output are completely separated.

The workflow file lives at .github/workflows/deploy.yml. When you push to the portfolio branch, the workflow spins up an Ubuntu machine, installs the correct version of Hugo, runs hugo --minify to build the site, and then deploys the output to GitHub Pages. The whole process takes about ninety seconds.

There was one non-obvious configuration step that caught me off guard. GitHub Pages has environment protection rules that restrict which branches are allowed to trigger deployments. By default only the main branch is allowed. Since my working branch is named portfolio, the first deployment attempt failed with a message saying the branch was not authorized. The fix was to go into the repository settings, find the github-pages environment, and explicitly add portfolio as an allowed deployment branch.

What I Learned

This project taught me a few things I will carry into future web projects.

Separating content from presentation is genuinely valuable. Because all my blog posts and project pages are Markdown files, I can add new content without touching any HTML or CSS. I can also completely redesign the visual appearance of the site without touching a single piece of content. Those two concerns being separate is exactly how it should be.

Build-time solutions are better than runtime solutions when you have the choice. Every time I tried to solve a problem with client-side JavaScript, I eventually found a cleaner solution that ran at build time. Hugo's table of contents is a good example. Solving it in the browser created timing problems and complexity. Solving it during the build process eliminated both.

CSS variables make theming much simpler. Before this project I had never built a complete dark and light theme. I expected it to be significantly more work than it was. Because I defined the entire color palette as CSS variables from the beginning, switching themes is a single class change on the body element.

Testing locally before pushing is obvious advice but it saved me many times. The GitHub Actions deployment takes ninety seconds per push. If you push a broken version, you wait ninety seconds to find out it is broken, fix it locally, and wait another ninety seconds. Running hugo server -D locally before every push kept that feedback loop tight.

Where Things Stand Now

The site is live, the blog is active, and the deployment pipeline works reliably. Every time I push to the portfolio branch, the site updates automatically within a couple of minutes. Adding a new blog post is as simple as running hugo new blog/post-title.md, writing in Markdown, and pushing to GitHub.

The codebase is clean and understandable. There are no build tools beyond Hugo itself, no npm dependencies, no webpack configuration. Just HTML templates, a CSS file, a JavaScript file, and Markdown content. A year from now I will be able to open this project and understand exactly how every part of it

works.

If you are thinking about building your own portfolio and are unsure whether to use Hugo or some other tool, my honest answer is that Hugo is worth the initial learning curve if you plan to have a blog. The content management experience once everything is set up is genuinely pleasant. If you just want a single page with no blog, a well-written HTML file is probably all you need.

The source code for this site is available on my GitHub if you want to see how anything specific is implemented.

Related Reading

These two posts cover topics that connect directly to what was built here.

You Don't Need to Be a Programmer to Be a Tech Person (/blog/tech-survival-guide-2026) The broader technical foundation behind things like Git, CI/CD, and the command line that this portfolio build relied on. If any part of the deployment workflow felt unfamiliar, this guide breaks down all twelve fundamentals from scratch, no degree required.

Hypervisors, KVM & QEMU: A Complete Guide to Virtual Machines on Ubuntu 24.04 LTS (/blog/hypervisors-kvm-qemu-complete-guide) After getting the portfolio running, this was the next big technical project I documented. A full guide to setting up KVM/QEMU on Ubuntu, installing Windows 11, Kali Linux, and Parrot OS as VMs, and solving the Secure Boot + NVIDIA driver problem that most guides skip entirely.